

Introduction à HTML et CSS

Objectifs du cours

- comprendre le rôle de HTML et de CSS dans la création d'une page web ;
- construire la structure d'un document HTML valide ;
- utiliser les principales balises HTML pour organiser et enrichir le contenu ;
- créer des liens, intégrer des images, des listes et des tableaux ;
- structurer une page avec les éléments sémantiques de HTML5 ;
- appliquer une feuille de style CSS à une page HTML ;
- utiliser les principales propriétés CSS pour améliorer la lisibilité et l'apparence d'une page ;
- réaliser un mini-site statique simple, structuré et cohérent.

Séance	Compétences principales
S1 – HTML	C05 – Concevoir un système informatique C08 – Coder
S2 – HTML/CSS	C08 – Coder C06 – Valider un système informatique

Séance 1 — HTML : structurer et enrichir une page web

Compétences ciblées :

- **C05 – Concevoir un système informatique**
 - **C08 – Coder**
- analyser le contenu et la structure attendue d'une page ;
 - organiser les informations ;
 - concevoir une structure HTML cohérente ;
 - produire le code HTML ;
 - utiliser les éléments sémantiques adaptés.

Les activités/tâches :

- **D1 – Élaboration et appropriation d'un cahier des charges**
 - D1-T1 : collecte des informations — 
 - D1-T2 : analyse des informations — 
 - D1-T3 : interprétation d'un cahier des charges — 
- **D2 – Développement et validation de solutions logicielles**
 - D2-T1 : conception de l'architecture d'une solution logicielle — 
 - D2-T3 : développement, utilisation ou adaptation de composants logiciels — 

Partie 1 — Découvrir HTML

1.1. Rôle de HTML et différence avec CSS

- **HTML** (*HyperText Markup Language*) : langage de balisage pour **structurer** le contenu d'une page web (titres, paragraphes, liens, images, etc.).
- **CSS** (*Cascading Style Sheets*) : langage pour **mettre en forme** le contenu (couleurs, polices, marges, etc.).

HTML structure le contenu, CSS contrôle sa présentation.

Le fichier HTML porte généralement l'extension **.html**.

1.2. Structure minimale d'un document HTML

Un document HTML valide doit contenir :

- **<!DOCTYPE html>** : déclaration du type de document (HTML5).
- **<html lang="fr">** : racine du document, avec attribution de la langue.
- **<head>** : en-tête (métadonnées : titre, encodage, etc.).
- **<body>** : corps (contenu affiché dans la page).

Exemple :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Ma première page</title>
  </head>
  <body>
    <!-- Contenu visible de la page -->
    <h1>Bonjour le monde !</h1>
    <p>Ceci est un paragraphe.</p>
  </body>
</html>
```

Remarques :

- HTML est **insensible à la casse**, mais les bonnes pratiques recommandent les balises en minuscules.
- Outils recommandés : Visual Studio Code, Sublime Text.

1.3. Encodage UTF-8 et `<title>`

- `<meta charset="utf-8">` : garantit l'affichage correct des caractères spéciaux (accents, symboles).
- `<title>` : titre de la page, affiché dans l'onglet du navigateur et dans les résultats de recherche.

1.4. Titres et paragraphes

- **Titres** : `<h1>` (principal) à `<h6>` (sous-titre).
- **Paragraphe** : `<p>`.

Exemple :

```
<h1>Titre principal</h1>
<h2>Sous-titre</h2>
<h3>Sous-section</h3>
<p>Un paragraphe avec <strong>du texte important</strong> et <em>en italique</em>.</p>
```

1.5. Les attributs HTML

Les attributs permettent de fournir des informations supplémentaires à un élément.

Exemple :

```
<a href="https://carnus.fr">Visiter le site</a>
```

Ici :

- `href` est un attribut ;
- `"https://carnus.fr"` est sa valeur.

La syntaxe générale est :

```
<balise attribut="valeur">
```

TP 1 — Créer une première page personnelle

Objectif : Créer un fichier `index.html` avec :

- Structure minimale (doctype, html, head, body, title).
- Un titre `<h1>` avec votre prénom et nom.
- Un paragraphe `<p>` vous décrivant.
- Un titre `<h2>` pour une section "À propos".

Résultat attendu : Une page web valide, affichée correctement dans le navigateur.

Partie 2 — Enrichir le contenu

2.1. Mise en forme sémantique

- `<strong>` : texte **important** (gras par défaut).
- `<em>` : texte mis *en avant* (italique par défaut).

Exemple :

```
<p>  
  Ce cours est <strong>important</strong> pour la suite du <em>projet</em>.  
</p>
```

2.2. Liens hypertextes

- **Lien externe :** `<a href="https://carnus.fr">Visiter le site</a>`
- **Lien interne :** `<a href="page2.html">Aller à la page 2</a>`
- **Lien e-mail :** `<a href="mailto:lycee@carnus.fr">Envoyer un e-mail</a>`

L'attribut `href` indique la destination du lien.

Exemple :

```
<nav>  
  <a href="https://fr.wikipedia.org">Wikipédia</a> |
```

```
<a href="mailto:contact@carnus.fr">Nous contacter</a>
</nav>
```

2.3. Images

- **Balise** : ``
- **Attributs** :
 - `src` : chemin vers l'image (relatif ou absolu).
 - `alt` : description alternatif (accessibilité).
 - `width/height` : dimensions (en pixels ou %).
 - `title` : infobulle au survol (information supplémentaire éventuellement affichée au survol).

Lorsque `width` et `height` sont utilisés ensemble, il faut respecter les proportions de l'image afin d'éviter sa déformation.

Exemple :

```

```

Astuce :

Pour comprendre l'utilité de `alt`, essayez volontairement de modifier le nom du fichier dans `src` :

```

```

Le navigateur ne pourra pas charger l'image, mais le texte alternatif pourra être affiché.

TP 2 — Ajouter liens et images à la page

Objectif : Compléter votre page personnelle.

À partir de la page personnelle créée précédemment :

- ajouter une image ;
- renseigner correctement ses attributs `alt` et `title` ;
- ajouter un lien vers un site externe (ex. : Wikipedia) ;
- ajouter un lien vers une deuxième page HTML ;
- ajouter un lien `mailto:` vers votre adresse e-mail.

Partie 3 — Organiser l'information

3.1. Listes

- **Liste non ordonnée** : `<ul>` + `<li>` (puces).
- **Liste ordonnée** : `<ol>` + `<li>` (numérotée).

Exemple :

```
<h3>Mes compétences</h3>
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<h3>Étapes à suivre</h3>
<ol>
  <li>Créer la structure HTML</li>
  <li>Ajouter du style CSS</li>
</ol>
```

3.2. Tableaux

Les tableaux permettent de présenter des données organisées en lignes et en colonnes.

Les principaux éléments sont :

- `<table>` : tableau ;
- `<thead>` : en-tête du tableau ;
- `<tbody>` : corps du tableau ;
- `<tr>` : ligne ;
- `<th>` : cellule d'en-tête ;
- `<td>` : cellule de données.

Exemple :

```
<table>
  <thead>
    <tr>
      <th>Matière</th>
      <th>Note</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>Informatique</td>
      <td>16</td>
    </tr>
    <tr>
      <td>Réseaux</td>
```

```
<td>14</td>
</tr>
</tbody>
</table>
```

La présentation du tableau sera réalisée avec CSS lors de la deuxième séance.

TP 3 — Créer des listes et un tableau simple

Ajouter à votre page personnelle :

- créer une liste non ordonnée contenant 3 à 5 matières ou compétences ;
- créer une liste ordonnée présentant les étapes d'une activité ;
- créer un tableau simple avec 3 colonnes (Matière, Note 1, Note 2) et 2 lignes.

Partie 4 — Structurer une page avec HTML5

4.1. Notion de sémantique HTML

- **Pourquoi ?** : Améliorer l'accessibilité, le référencement (SEO), et la lisibilité du code.
- **Différence** :
 - `<div>` : conteneur générique (sans signification).
 - Balises sémantiques : `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>` : décrivent le rôle de chaque partie.

4.2. `<div>` et éléments sémantiques

`<div>` est un conteneur générique qui ne possède pas de signification particulière.

Il peut être utile lorsqu'aucun élément sémantique ne correspond au besoin.

Par exemple :

```
<div class="bloc">
...
</div>
```

Lorsque le rôle du contenu est identifiable, il est préférable d'utiliser un élément sémantique adapté :

```
<article>
...
</article>
```

La sémantique améliore notamment la compréhension du document par les développeurs, les outils et les technologies d'assistance.

4.3. Balises sémantiques HTML5

Balises sémantiques HTML5

Balise	Rôle
<code><header></code>	En-tête de la page ou section
<code><nav></code>	Barre de navigation
<code><main></code>	Contenu principal
<code><section></code>	Section thématique
<code><article></code>	Contenu autonome (ex. : article de blog)
<code><aside></code>	Contenu secondaire (ex. : barre latérale)
<code><footer></code>	Pied de page

4.4. Organisation logique d'une page

Une page peut être organisée de la manière suivante :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Mon site</title>
</head>

<body>
  <header>
    <h1>Bienvenue sur mon site</h1>
  </header>

  <nav>
    <ul>
      <li><a href="#">Accueil</a></li>
      <li><a href="#">À propos</a></li>
    </ul>
  </nav>

  <main>
    <section>
      <h2>Présentation</h2>

      <article>
        <h3>Mon parcours</h3>
        <p>Voici une présentation de mon parcours.</p>
      </article>
    </section>

    <aside>
```



```
<h2>Informations complémentaires</h2>
<p>Quelques informations supplémentaires.</p>
</aside>
</main>

<footer>
  <p>&copy; 2026 Prénom Nom</p>
</footer>

</body>
</html>
```

La mise en page visuelle de cette structure sera réalisée avec CSS lors de la séance 2.

TP 4 — Construire une page structurée

Créer un nouveau fichier nommé `page1.html` contenant au minimum :

- `<header>` + `<h1>`.
- `<nav>` avec 3 liens.
- `<main>` contenant :
 - Un `<article>` avec un titre et un paragraphe.
 - Un `<aside>` avec une liste de liens.
- `<footer>` avec un copyright.

La page doit également contenir au moins :

- un titre ;
 - un paragraphe ;
 - une image ;
 - une liste ;
 - un lien.
-

Partie 5 — Projet HTML

Objectif : Réaliser une page ou un mini-site **sans CSS**, en réinvestissant les notions étudiées pendant la séance.

Le projet peut prendre différentes formes :

- présentation personnelle ;
- page consacrée à un hobby ;
- présentation d'un club ;
- fiche de cours ;
- présentation d'une formation ;
- sujet libre validé par l'enseignant.

Contraintes

Le projet devra comporter, selon sa nature :

- une structure HTML5 complète ;
- des titres hiérarchisés ;
- plusieurs paragraphes ;
- au moins une image avec un `alt` pertinent ;
- au moins une liste ;
- au moins un lien ;
- des éléments sémantiques HTML5 ;
- éventuellement un tableau ;
- une navigation entre plusieurs pages si cela est pertinent.

Valider le code avec [le validateur W3C](#)

Rendu attendu : Une page web complète, structurée, lisible et accessible.

Bilan de la séance 1

Savoir construire une page web complète, structurée et lisible en HTML. Le contenu est organisé, mais la présentation graphique reste volontairement simple.

Séance 2 — HTML/CSS : mettre en forme et améliorer une page web

Compétences ciblées :

- C08 – Coder
- C06 – Valider un système informatique

Développer → Tester → Corriger → Valider

Ecrire du CSS, expérimenter différentes mises en forme, observer le rendu, identifier les écarts avec les attentes et corriger.

Activités/tâches :

- D2 – Développement et validation de solutions logicielles
 - D2-T3 : développement, utilisation ou adaptation de composants logiciels — 
 - D2-T4 : tests de mise en production — 
 - D2-T5 : recette et validation — 

Partie 1 — Découvrir CSS

1.1. Rôle de CSS

CSS (*Cascading Style Sheets*) est le langage utilisé pour définir la **présentation visuelle** d'une page HTML.

HTML permet notamment de dire :

```
<h1>Mon site</h1>
<p>Bienvenue sur mon site.</p>
```

CSS permet ensuite de définir l'apparence de ces éléments :

```
h1 {
  color: darkblue;
}

p {
  font-size: 18px;
}
```

HTML structure le contenu, CSS contrôle son apparence.

Cette séparation entre contenu et présentation permet de modifier l'apparence d'un site sans devoir réécrire toute sa structure HTML.

- **Séparer le contenu (HTML) de la présentation (CSS).**
- **Avantages :**
 - Maintenance plus facile.
 - Réutilisation du style sur plusieurs pages.
 - Adaptabilité (responsive design).

1.2. Intégrer CSS dans HTML

3 méthodes (par ordre de priorité) :

1. Feuille de style externe (recommandé) :

Pour un projet comportant plusieurs pages, il est recommandé d'utiliser une **feuille de style externe**.

Par exemple :

```
mon-site/  
├── index.html  
├── page2.html  
├── style.css  
└── images/  
    └── photo.jpg
```

Dans le fichier HTML :

```
<head>  
  <meta charset="utf-8">  
  <title>Mon site</title>  
  <link rel="stylesheet" href="style.css">  
</head>
```

Dans **style.css** :

```
body {  
  background-color: #f0f0f0;  
  color: #333;  
  font-family: Arial, sans-serif;  
}
```

Une même feuille de style peut ainsi être utilisée par plusieurs pages.

2. Balise **<style>** dans **<head>** :

```
<style>  
  body { background-color: #f0f0f0; }
```

```
</style>
```

3. Attribut **style** (à éviter) :

```
<p style="color: blue;">Texte bleu</p>
```

1.3. Sélecteurs CSS de base

Un sélecteur permet de choisir les éléments auxquels une règle CSS doit s'appliquer.

- **Sélecteur d'élément** : `p { color: red; }` (cible tous les `<p>`).
- **Sélecteur de classe** : `.ma-classe { ... }`.
- **Sélecteur d'ID** : `#mon-id { ... }`.

Exemple :

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f9f9f9;  
}  
  
h1 {  
  color: #333;  
  text-align: center;  
}  
  
p {  
  font-size: 18px;  
}
```

- `body` sélectionne l'élément `<body>` ;
- `h1` sélectionne les titres `<h1>` ;
- `p` sélectionne les paragraphes.

Une règle CSS possède généralement la forme :

```
selecteur {  
  propriete: valeur;  
}
```

1.4. Propriétés CSS essentielles

- **Couleurs** :
 - `color` : couleur du texte.
 - `background-color` : couleur de fond.

- **Polices :**
 - `font-family` : police utilisée (ex. : `Arial, sans-serif`).
 - `font-size` : taille du texte (ex. : `16px`).
- **Texte :**
 - `text-align` : alignement (`left, center, right`).

TP 1 — Appliquer une première identité visuelle

Reprendre votre projet HTML et ajouter :

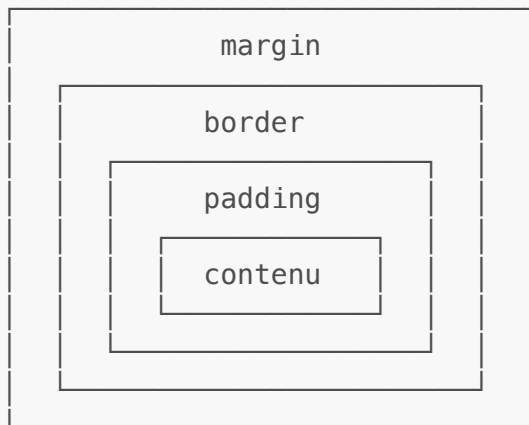
- Une feuille de style externe `styles.css`.
- Un fond de page (`background-color`).
- Une police personnalisée (`font-family`).
- Une couleur pour les titres (`h1, h2`).
- modifier la taille des titres et des paragraphes.

Partie 2 — Comprendre le modèle de boîte (Box Model)

2.1. Principe du Box Model

En CSS, chaque élément est considéré comme une **boîte**.

Le modèle de boîte comprend notamment :



- **Contenu** (content).
- **Remplissage** (`padding`) : espace entre le contenu et la bordure.
- **Bordure** (`border`) : contour de la boîte.
- **Marge** (`margin`) : espace entre la boîte et les autres éléments (espace extérieur).

2.2. Propriétés du Box Model

- `width / height` : dimensions du contenu.
- `padding` : espace intérieur (ex. : `padding: 10px;`).
- `border` : bordure (ex. : `border: 1px solid black;`).

- **margin** : espace extérieur (ex. : **margin: 20px;**).

Exemples :

```
div {  
  width: 200px;  
  padding: 15px;  
  border: 2px solid blue;  
  margin: 10px;  
}
```

```
article {  
  width: 500px;  
  padding: 20px;  
  border: 2px solid #333;  
  margin: 30px;  
}
```

Le **padding** crée un espace entre le contenu et la bordure.

Le **margin** crée un espace entre la boîte et les éléments environnants.

Mini-TP — Expérimenter avec plusieurs boîtes

Créer 3 éléments ou `<div>` avec :

- Des largeurs et hauteurs différentes.
- Des **padding**, **border**, et **margin** variés.
- Observer l'impact sur l'affichage.

Partie 3 — Styliser textes, liens et images

3.1. Mise en forme du texte

- **font-size** : taille de la police.
- **line-height** : hauteur de ligne.
- **text-align** : alignement.
- **font-weight** : épaisseur (**bold**, **normal**).

Exemple :

```
p {  
  font-size: 18px;  
  line-height: 1.6;
```

```
text-align: justify;
}
```

3.2. Styliser les liens

- **État par défaut** : `a { color: blue; }.`
- **Au survol** : `a:hover { color: red; }.`
- **Visité** : `a:visited { color: purple; }.`

Exemple :

```
a {
  color: #0066cc;
  text-decoration: none;
}

a:hover {
  color: #ff6600;
  text-decoration: underline;
}
```

3.3. Styliser les images

- **border** : bordure.
- **border-radius** : coins arrondis.
- **display: block** : centrer avec `margin: 0 auto;`.

Exemples :

```
img {
  width: 300px;
  border: 3px solid #ddd;
  border-radius: 10px;
  display: block;
  margin: 0 auto;
}
```

```
img {
  max-width: 100%;
  height: auto;
}
```

afin d'éviter qu'une image dépasse de son conteneur sur les petits écrans.

TP 2 — Améliorer le projet

Modifier la feuille de style pour :

- améliorer la lisibilité des paragraphes ;
- modifier l'apparence des liens ;
- ajouter un effet `:hover` ;
- mettre en valeur les images ;
- ajouter des bordures ou des coins arrondis ;
- harmoniser les couleurs et les tailles de texte.

Partie 4 — Styliser listes et tableaux

4.1. Les listes

- **list-style-type** : type de puce (`disc`, `circle`, `square`, `none`).
- **list-style-position** : position des puces (`inside`, `outside`).

Exemples :

```
ul {  
  list-style-type: square;  
  padding-left: 20px;  
}
```

Pour supprimer les marqueurs :

```
ul {  
  list-style-type: none;  
}
```

Les listes peuvent également être espacées avec `margin` et `padding`.

4.2. Les tableaux

- **border-collapse** : `collapse` ; fusionner les bordures adjacentes du tableau.
- **border-spacing** : espace entre les bordures.
- **Couleurs** : `background-color` pour les cellules.

Exemple :

```
table {  
  width: 100%;  
  border-collapse: collapse;  
}
```

```
th, td {  
  border: 1px solid #ddd;  
  padding: 8px;  
  text-align: left;  
}  
  
th {  
  background-color: #f2f2f2;  
}
```

On peut également mettre en évidence certaines cellules :

```
td {  
  background-color: #fafafa;  
}
```

TP 3 — Finaliser les listes et tableaux

Dans le projet :

- améliorer l'apparence des listes ;
- ajouter des espacements ;
- mettre en forme le tableau ;
- modifier les couleurs de l'en-tête ;
- ajouter des bordures ;
- améliorer la lisibilité des cellules.

Partie 5 — Projet final HTML/CSS

5.1. Objectif

Reprendre le projet HTML réalisé lors de la séance 1 et le transformer en un **mini-site statique propre, clair, structuré et esthétique**.

Travail demandé

Le projet devra comporter :

- une structure HTML5 cohérente ;
- une navigation fonctionnelle ;
- une feuille CSS externe ;
- une identité visuelle cohérente ;
- des titres correctement hiérarchisés ;
- des paragraphes lisibles ;
- des liens stylisés ;
- des images correctement intégrées ;

- des listes mises en forme ;
- un tableau stylisé s'il est présent ;
- des espacements et des bordures cohérents.

5.2. Suggestions pour aller plus loin

- Ajouter un **formulaire de contact** (`<form>`).
- Utiliser des **polices Google Fonts**.
- Rendre le site **responsive** (adapté aux mobiles).

5.3. Rendu final

Le rendu consiste en un **mini-site statique HTML/CSS** comprenant une ou plusieurs pages.

Le site doit être :

- **propre** ;
- **clair** ;
- **structuré** ;
- **lisible** ;
- **cohérent visuellement**.

Bilan de la Séance 2

- Séparer contenu (HTML) et présentation (CSS).
- Utiliser les sélecteurs et propriétés CSS de base.
- Appliquer le Box Model pour organiser les éléments.
- Styliser textes, liens, images, listes et tableaux.

Résultat : Un mini-site statique complet, esthétique et fonctionnel.

Annexes

Ressources utiles

- **Validation HTML** : <https://validator.w3.org>
 - **Validation CSS** : <https://jigsaw.w3.org/css-validator>
 - **Polices Google Fonts** : <https://fonts.google.com>
 - **Éditeurs de code** : Visual Studio Code, Sublime Text.
-

TP — Maquette d'un mini-site

Objectif

Créer un **mini-site personnel** ou un **portfolio** en HTML, puis le mettre en forme avec CSS.

Étape 1 — Version HTML

Le site doit comporter :

- un en-tête avec le nom du site ;
- un menu de navigation ;
- un contenu principal ;
- un ou plusieurs articles ou sections ;
- une barre latérale avec `<aside>` ;
- un pied de page ;
- au moins une image ;
- au moins un lien externe.

Étape 2 — Version CSS

Créer un fichier `style.css` permettant notamment de :

- choisir les couleurs ;
- définir les polices ;
- gérer les espacements ;
- styliser les titres ;
- mettre en forme les liens ;
- styliser les images ;
- mettre en forme les listes ;
- mettre en forme les tableaux s'ils sont présents.

Exemple de projet final

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Mon Portfolio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Jean Dupont</h1>
    <nav>
      <ul>
        <li><a href="#">Accueil</a></li>
        <li><a href="#">Projets</a></li>
        <li><a href="#">Contact</a></li>
```

```
        </ul>
    </nav>
</header>
<main>
    <article>
        <h2>À propos</h2>
        
        <p>Je suis étudiant en développement web...</p>
    </article>
    <aside>
        <h3>Compétences</h3>
        <ul>
            <li>HTML/CSS</li>
            <li>JavaScript</li>
        </ul>
    </aside>
</main>
<footer>
    <p>&copy; 2026 Charles Carnus</p>
</footer>
</body>
</html>
```

```
/* styles.css */
body {
    font-family: 'Arial', sans-serif;
    line-height: 1.6;
    margin: 0;
    padding: 0;
    background-color: #f9f9f9;
    color: #333;
}

header {
    background-color: #333;
    color: white;
    padding: 1rem;
    text-align: center;
}

nav ul {
    list-style-type: none;
    padding: 0;
}

nav li {
    display: inline;
    margin-right: 1rem;
}

a {
```

```
    color: white;
    text-decoration: none;
}

a:hover {
    text-decoration: underline;
}

main {
    display: flex;
    padding: 1rem;
}

article {
    flex: 3;
    padding: 1rem;
}

aside {
    flex: 1;
    background-color: #e6e6e6;
    padding: 1rem;
}

footer {
    text-align: center;
    padding: 1rem;
    background-color: #333;
    color: white;
}
```
